

PRÁTICA 3: ASSEMBLY (rotinas, gestão de pilha e recursividade)

I. INSTRUÇÕES

- Na aula de apresentação, os alunos deverão explicar as principais decisões tomadas durante a codificação e executar os programas de modo a mostrar o funcionamento correto de cada um e/ou possíveis fontes de erros.
- Arquivo compactado nomeado "praticaX_nome_aluno.zip", onde X representa o número da prática entregue.
- Esse arquivo deve conter uma pasta com todos os arquivos de código-fonte dos programas em assembly gerados (*.asm) e o relatório (*.pdf). Após o upload do arquivo, faça o download e certifique-se de que o arquivo descompacta e que os arquivos são abertos corretamente.
- Também deverá ser entregue um arquivo *.pdf com imagens de tela da execução dos programas assembly mostrando o funcionamento correto de cada um. Escreva também um ou dois parágrafos explicando as principais decisões tomadas e o que deve ser evidenciado nas imagens de tela.

II. ATENÇÃO

- Todos os programas devem ser desenvolvidos usando **apenas** o conjunto de instruções do MIPS apresentado a seguir. Sim, é possível 😊

MIPS assembly language				
Category	Instruction	Example	Meaning	Comments
Arithmetic	add	add \$s1, \$s2, \$s3	\$s1 = \$s2 + \$s3	Three operands; data in registers
	subtract	sub \$s1, \$s2, \$s3	\$s1 = \$s2 - \$s3	Three operands; data in registers
	add immediate	addi \$s1, \$s2, 100	\$s1 = \$s2 + 100	Used to add constants
Data transfer	load word	lw \$s1, 100(\$s2)	\$s1 = Memory[\$s2 + 100]	Word from memory to register
	store word	sw \$s1, 100(\$s2)	Memory[\$s2 + 100] = \$s1	Word from register to memory
Conditional branch	branch on equal	beq \$s1, \$s2, 25	if (\$s1 == \$s2) go to PC + 4 + 100	Equal test; PC-relative branch
	branch on not equal	bne \$s1, \$s2, 25	if (\$s1 != \$s2) go to PC + 4 + 100	Not equal test; PC-relative
	set on less than	slt \$s1, \$s2, \$s3	if (\$s2 < \$s3) \$s1 = 1; else \$s1 = 0	Compare less than; for beq, bne
	set less than immediate	slti \$s1, \$s2, 100	if (\$s2 < 100) \$s1 = 1; else \$s1 = 0	Compare less than constant
Unconditional jump	jump	j 2500	go to 10000	Jump to target address
	jump register	jr \$ra	go to \$ra	For switch, procedure return
	jump and link	jal 2500	\$ra = PC + 4; go to 10000	For procedure call

III. UM POUCO DE TEORIA

A construção de um programa pode ser simplificada quando o problema a resolver é dividido em “problemas menores” por um mecanismo de chamada de procedimentos (ou sub-rotinas, como são conhecidos em Assembly). As sub-rotinas permitem escrever um programa complexo de uma forma modular, permitindo assim uma melhor depuração de erros e também a reutilização de sub-rotinas ao longo de todo o programa.

Quando um procedimento chama outro procedimento, o controle é transferido do procedimento que chama para o procedimento chamado. As instruções **jal** e **jr** tornam possível o correto funcionamento deste mecanismo. Para executar um procedimento quatro passos são necessários:

1. Guardar o endereço de retorno. O endereço de retorno é o endereço da instrução que segue imediatamente após o ponto da chamada.
2. Chamar o procedimento.
3. Executar o procedimento.
4. Recuperar o endereço de retorno e retornar ao ponto da chamada.

A arquitetura do MIPS fornece duas instruções que são utilizadas em conjunção para realizar a chamada e retornar:

jal nome_do_procedimento *#jump and link*

Esta instrução guarda no registo **\$ra** o endereço da instrução que segue imediatamente após o **jal** (endereço de retorno = PC + 4) e salta para o procedimento cujo rótulo é especificado. Esta instrução implementa os passos 1 e 2. Para o retorno (passo 4), utiliza-se a instrução abaixo, a qual salta para o endereço armazenado no registo **\$ra**. Durante a execução o conteúdo de **\$ra** substitui o conteúdo do registo PC.

jr \$ra *#jump register*

Sabendo que a instrução **jal** guarda o endereço de retorno no registrador **\$ra**, vamos assumir agora que um procedimento pretende chamar um procedimento, então, o programador tem de guardar o valor de **\$ra** “em algum lugar” porque senão o novo **jal** irá escrever por cima do seu valor. Uma vez que o número de chamadas não tem limite (procedimentos podem chamar outros procedimentos que também chamam procedimentos etc.) e não podemos utilizar uma localização fixa (seja ela um registrador ou uma posição de memória) para guardar os valores de retorno **\$ra**.

Por esse motivo, necessitamos de uma estrutura dinâmica, algo que nos permita armazenar dados, mas sem escrever por cima desses dados previamente guardados. Como sabemos que o último dado guardado será o primeiro que iremos utilizar (o último valor de **\$ra** guardado será o primeiro a ser utilizado quando houver o retorno), utilizamos uma estrutura chamada **pilha (stack)**.

A arquitetura do MIPS descreve uma pilha cujo topo começa por ter um valor elevado (0x7fff ffff) na memória principal e que cresce para baixo, isto é, sempre que se coloca algum registo na pilha, o endereço do topo do **stack** decresce. O registrador **\$sp** (stack pointer) aponta sempre – se o programa estiver correto – para o topo da pilha, isto é, para a primeira posição vazia na pilha.

Como a pilha é apenas um “pedaço” da memória principal, iremos acessá-la usando as mesmas instruções de load/store que usamos para acessar a memória de dados. Por exemplo, para guardar o valor de um registrador na pilha:

```
sub $sp,$sp,4    # primeiro atualize o stack pointer
sw $ra, 4($sp)   # armazene o valor do registrador para na pilha
```

Começamos por subtrair a quantidade correta ao endereço do **stack pointer**, uma vez que a pilha vai aumentando para baixo (na memória principal). Neste exemplo, foi subtraído 4 porque queremos guardar uma palavra (4 bytes) que está no registro **\$ra**.

Depois, fazemos o armazenamento (**store**) usando um deslocamento positivo: 4(\$sp) significa o endereço contido em \$sp + 4, a fim de apontarmos para a posição de memória vazia para onde o \$sp apontava antes de o modificarmos. Esta operação também é chamada de **push**.

Para recuperar um valor previamente guardado na pilha, a operação inversa chamada **pop**:

```
lw $ra,4($sp)    # recupera o dado da pilha
add $sp,$sp,4     # atualiza o $sp: encolhe-se o stack em uma palavra
```

Para guardar duas palavras repete-se a operação e, assim sucessivamente:

```
sub $sp,$sp,8     # queremos guardar 2 palavras (8 bytes)
sw $s0,8($sp)
sw $s1,4($sp)
```

Existem duas formas de passar parâmetros a um procedimento: pelos registradores e pela pilha. A convenção de utilização dos registradores no MIPS especifica que os 4 primeiros argumentos de um procedimento são passados por registradores (**\$a0-\$a3**). Os restantes são passados pela pilha. Passar parâmetros pelos registradores é eficiente porque evita acessos à memória. Se o procedimento chamar outros procedimentos, então os registradores que contiverem parâmetros têm de ser guardados (\$a0-\$a3 não são preservados ao longo das chamadas). Isso pode ser feito:

- Nos registradores que são preservados ao longo das chamadas (\$s0-\$s7);
- Na pilha.

Já o valor de retorno de um procedimento deve ser armazenado em **\$v0**.

Exemplo 1:

<pre>int example (int g, int h, int i, int j) { int f; f = (g + h) - (i + j); return f; }</pre> Suponha que g, h, i, j passados como parâmetro correspondam aos registradores \$a0, \$a1, \$a2 e \$a3 e o resultado f em \$s0	<pre>example: sub \$sp, \$sp, 12 # ajusta a pilha sw \$t1, 8(\$sp) # salva \$t1 sw \$t0, 4(\$sp) # salva \$t0 sw \$s0, 0(\$sp) # salva \$s0 add \$t0, \$a0, \$a1 # \$t0 = g + h add \$t1, \$a2, \$a3 # \$t1 = i + j sub \$s0, \$t0, \$t1 # f = \$t0 - \$t1 add \$v0, \$s0, \$zero # retorna f lw \$s0, 0(\$sp) # restaura \$s0 lw \$t0, 4(\$sp) # restaura \$t0 lw \$t1, 8(\$sp) # restaura \$t1 add \$sp, \$sp, 12 # remove pilha jr \$ra</pre>
--	--

Exemplo 2: procedimento recursivo

<pre>int soma (int n, int acc) { if (n > 0) return soma(n-1, acc + n); else return acc; }</pre>	<pre>soma: slti \$t0, \$a0, 1 # testa se n<=0 bne \$t0, \$zero, exit # desvia se n<=0 add \$a1, \$a1, \$a0 # soma n e acc subi \$a0, \$a0, 1 # n -1 j soma # desvia para soma exit: addi \$v0, \$a1, \$zero # resultado jr \$ra # retorna</pre>
Suponha que n, acc passadas como parâmetro correspondam aos registradores \$a0, \$a1	

IV. APRESENTAR NA AULA DO DIA 04/09/2026 – NÃO É NECESSÁRIO ENTREGAR CÓDIGO E RELATÓRIO PARA ESSA PARTE

- 1) Dada a implementação em Portugol da função abaixo, transcreva-a para a linguagem Assembly MIPS. Sua implementação deve utilizar a pilha para preservar os valores dos parâmetros e dos resultados intermediários necessários durante as chamadas recursivas. Após implementar a função, execute o programa e determine os valores retornados para H(2), H(5) e H(8).

```
função H(N : natural) : natural  
início  
    se N < 3 então  
        retorne 2 * N + 1  
    senão retorne H(N - 2) + 2 * H(N - 3) + N  
fim
```

V. ENTREGAR CÓDIGO E RELATÓRIO NO MOODLE ATÉ 10/09/2026 ÀS 23H59

- 2) Dada a implementação em Portugol da função abaixo, transcreva-a para a linguagem Assembly MIPS. Execute o programa e mostre quais são os valores de F(3), de F(7) e F(15).

```
função F(N : natural) : natural  
início  
    se N < 4 então  
        retorne 3 * N  
    senão retorne 2 * F(N - 4) + 5  
fim
```

- 3) Dada a implementação em portugol a seguir, implemente recursivamente uma função Max que retorne o maior valor armazenado em um vetor V, contendo N números inteiros armazenados em memória (mínimo de 10 elementos para testes).

```

funcao inteiro Max(inteiro V[], inteiro n)
inicio
    se (n = 1) entao
        retorne V[1]
    fimse

    inteiro maxAnterior
    maxAnterior <- Max(V, n - 1)

    se (V[n] > maxAnterior) entao
        retorne V[n]
    senao
        retorne maxAnterior
    fimse
fimfuncao

```

- 4) O programa abaixo apresenta uma função recursiva que imprime (em ordem decrescente) os números de 1 a um número passado como parâmetro. Transcreva o código em linguagem de alto nível para linguagem Assembly MIPS.

```

int imp_ordem_decrescente( int n ) {
    if ( n > 0 ) {
        printf( "%d ", n );
        imp_ordem_decrescente( n-1 );
    }
}

```

- 5) O trecho de código a seguir, escrito em linguagem de alto nível, apresenta a Sequência de Fibonacci. Na matemática, trata-se de uma sequência de números inteiros, começando normalmente por 0 e 1, na qual, cada termo subsequente (número de Fibonacci) corresponde a soma dos dois anteriores. Transcreva esse programa para linguagem Assembly MIPS.

```

int fibonacci(int N) {
    if( N == 1 )
        return 1;
    else
        if( N == 2)    return 1;
    else
        return(fibonacci(N-1) + fibonacci(N-2));
}

main() {
    int num, F;
    printf("Entre com um número: ");    scanf("%d", &num);
    printf("A série de Fibonacci para %d elementos é:\n", num);

    for(F=1; F <= num; F++) {
        printf("%d, ", fibonacci(F));
    }
    printf("\nOk");
}

```